

Graph Algorithms - III

Minimum Spanning Trees

Given an undirected graph $G = (V, E)$ where every edge $(u, v) \in E$ has weight $w(u, v)$, we wish to find an acyclic subset $T \subseteq E$ that connects all vertices such that total weight

$$W(T) = \sum_{(u,v) \in T} w(u,v)$$

Spans

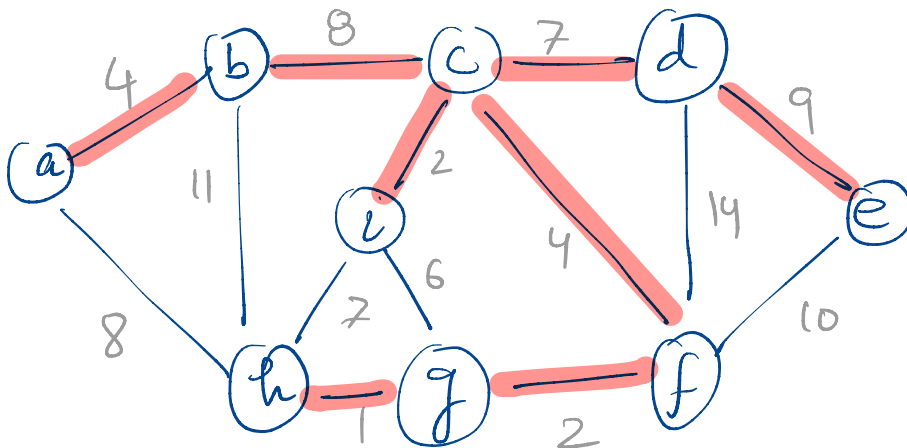
is minimized.

→ minimum weight

⇒ T is a tree

Such a tree is called a Minimum Spanning Tree (MST)

Example:



Algorithms:

- Kruskal's : $O(E \lg V)$ — bin heap.
- Prim's : $O(E \lg V)$ — bin-heap
- $O(E + V \lg V)$ — fib. heap

Greedy algorithms

↳ good if $|V| \ll |E|$

An algorithm that makes a choice while executing,
that looks best at that moment $\Rightarrow$ makes locally optimal choices

Does not guarantee optimal solution!

- A general strategy to find MST via greedy approach is:
 - Prior to each iteration, A is a subset of some MST
 - At each step we determine an edge (u, v) that can be added safely to A without violating the invariant
 - i.e. $A \cup \{(u, v)\}$ is a subset of MST
- safe edge

Generic - MST (G, w)

$A = \emptyset$

while A does not form a spanning tree

 | find an edge (u, v) that is safe for A

$A = A \cup \{(u, v)\}$

return A

Kruskal's algorithm

- Uses a disjoint-set data structure to maintain several disjoint sets of elements
- Each set contains vertices in one tree of the current forest
- Find-Set(u) returns a representative (identified) element from the set that contains u
- Union combines two sets

MST-Kruskal (G, w)

$$A = \emptyset$$

For each vertex $v \in G.V$ // create $|V|$ trees
Make-Set(v)

Create a list of edges in $G.E$

Sort the list into monotonically increasing
order by weight w

For each edge (u, v) taken from sorted list

$O(E \lg V)$

if Find-Set(u) $\neq$ Find-Set(v)

$A = A \cup \{(u, v)\}$

 Union(u, v)

return

Prim's algorithm

- Maintains a single tree A during execution & stops when A spans all vertices
- Each step adds a light-edge to A that connects A to an isolated vertex
- Maintains a min-priority queue Q of all vertices that are not in the tree based on the key attribute
- for each vertex v the attribute $v.key$ is the minimum weight of any edge connecting v to a vertex in the tree; $v.key = \infty$ if there is no such edge.

MST-PRIM (G, w, r) : r is arbitrary vertex

for each vertex $u \in G.V$

| $u.key = \infty$
| $u.\pi = NIL$

$r.key = 0$

$Q = \emptyset$

For each vertex $u \in G.V$

| Insert(Q, u)

while $Q \neq \emptyset$

$u = \text{Extract-min}(Q)$ // Add u to tree

For each vertex v in $G.Adj[u]$

if $v \in Q$ and $w(u, v) < v.key$

update keys
of u 's non-tree
neighbours

$v.\pi = u$

$v.key = w(u, v)$

Decrease-key($Q, u, w(u, v)$)

$O(E \lg V)$

— The algorithm implicitly maintains the set A as:
 $A = \{ (v, v.\pi) : v \in V - \{r\} - Q \}$ during execution,
and when the algorithm terminates:

$A = \{ (v, v.\pi) : v \in V - \{r\} \}$

→ MST